

|                                    |                                                                                          |                          |
|------------------------------------|------------------------------------------------------------------------------------------|--------------------------|
| <b>Professor:</b><br>Thiago Goveia | <b>Conteúdo:</b><br>Android: chamadas de API, acesso a recursos nativos e geração de APK | <b>Entrega:</b><br>06/07 |
|------------------------------------|------------------------------------------------------------------------------------------|--------------------------|

## 1. Entrega 3.2 (06/07)

Postar no SIGAA:

- Link do repositório contendo a Release (.apk), assim como foi feito para a versão em RN
- O APK deve conter:
  - No mínimo 2 telas, utilizando Jetpack Compose (Já feito na Entrega 01)
  - No mínimo 1 chamada de API (Especificar no [README.md](#) do repositório qual é esta chamada)
  - Acesso de no mínimo 1 recurso nativo (Especificar no [README.md](#) do repositório qual é e como acessar este recurso)

Nas seções a seguir são apresentados os materiais auxiliares.

## 2. Vídeos base

Os vídeos a seguir são sugestões para orientar o trabalho:

- ▶ Fazendo requisições HTTP em Apps Android com Retrofit | Jetpack Compose
- ▶ Image Capture From Camera With Jetpack Compose | Kotlin | Android Studio
- ▶ Como Gerar o APK do seu Aplicativo no Android Studio - Guia Iniciante

## 3. Consumo de API com Retrofit + Coroutines

O Retrofit é a biblioteca padrão de mercado para chamadas HTTP no Android nativo — equivalente ao fetch e à camada de serviço apresentada em React Native.

### 3.1 Dependências

```
// build.gradle.kts (app)
dependencies {
    implementation("com.squareup.retrofit2:retrofit:2.11.0")
    implementation("com.squareup.retrofit2:converter-gson:2.11.0")
    implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.0")
}
```

Atenção: Verificar as versões!

### 3.2 Modelo de Dados (Data Class)

Equivalente ao type/interface do TypeScript:

```
// model/Usuario.kt
data class Usuario(
    val id: Int,
    val name: String,
    val email: String
)
```

### 3.3 Interface da API

O Retrofit usa anotações para definir os endpoints — equivalente ao seu `services/usuarios.ts` do React Native:

```
// network/ApiService.kt
import retrofit2.http.GET
import retrofit2.http.POST
import retrofit2.http.Body
import retrofit2.http.Path

interface ApiService {
    @GET("users")
    suspend fun listarUsuarios(): List<Usuario>

    @GET("users/{id}")
    suspend fun buscarUsuarioPorId(@Path("id") id: Int): Usuario

    @POST("users")
    suspend fun criarUsuario(@Body usuario: Usuario): Usuario
}
```

#### A palavra-chave `suspend`

`suspend` marca uma função que pode ser pausada e retomada sem bloquear a thread principal. É o equivalente direto do `async` no TypeScript — só pode ser chamada de dentro de uma coroutine ou de outra função `suspend`.

### 3.4 Cliente Retrofit

```
// network/RetrofitClient.kt
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object RetrofitClient {
    private const val BASE_URL = "https://jsonplaceholder.typicode.com/"

    val api: ApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(ApiService::class.java)
    }
}
```

### 3.5 Estado de UI com Sealed Class

Equivalente ao padrão { data, loading, error } do useFetch — mas usando o sistema de tipos do Kotlin:

```
// ui/UiState.kt
sealed class UiState<out T> {
    object Loading : UiState<Nothing>()
    data class Success<T>(val data: T) : UiState<T>()
    data class Error(val mensagem: String) : UiState<Nothing>()
}
```

### 3.6 ViewModel — equivalente ao useFetch

```
// viewmodel/UsuariosViewModel.kt
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch

class UsuariosViewModel : ViewModel() {
    private val _uiState = MutableStateFlow<UiState<List<Usuario>>>(UiState.Loading)
    val uiState: StateFlow<UiState<List<Usuario>>> = _uiState.asStateFlow()

    init {
        buscarUsuarios()
    }

    fun buscarUsuarios() {
        viewModelScope.launch {
            _uiState.value = UiState.Loading
            try {
                val usuarios = RetrofitClient.api.listarUsuarios()
                _uiState.value = UiState.Success(usuarios)
            } catch (e: Exception) {
                _uiState.value = UiState.Error(e.message ?: "Erro desconhecido")
            }
        }
    }
}
```

#### ViewModel + StateFlow = useFetch

viewModelScope.launch → inicia uma coroutine vinculada ao ciclo de vida do ViewModel (cancelada automaticamente quando a tela é destruída, equivalente ao cleanup do useEffect com AbortController).

MutableStateFlow → equivale ao useState — mantém o valor e notifica observadores.

asStateFlow() → expõe a versão somente leitura, evitando que a UI altere o estado diretamente (boas práticas de encapsulamento).

### 3.7 Consumindo na Tela (Composable)

```
// ui/UsuariosScreen.kt
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.lifecycle.viewmodel.compose.viewModel

@Composable
fun UsuariosScreen(viewModel: UsuariosViewModel = viewModel()) {
    val uiState by viewModel.uiState.collectAsState()

    when (val estado = uiState) {
        is UiState.Loading -> {
            Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
                CircularProgressIndicator()
            }
        }
        is UiState.Error -> {
            Column(
                modifier = Modifier.fillMaxSize(),
                horizontalAlignment = Alignment.CenterHorizontally,
                verticalArrangement = Arrangement.Center
            ) {
                Text(text = estado.mensagem, color = Color.Red)
                Button(onClick = { viewModel.buscarUsuarios() }) {
                    Text("Tentar novamente")
                }
            }
        }
        is UiState.Success -> {
            LazyColumn(modifier = Modifier.padding(16.dp)) {
                items(estado.data) { usuario ->
                    Card(modifier = Modifier.fillMaxWidth().padding(vertical = 4.dp)) {
                        Column(modifier = Modifier.padding(12.dp)) {
                            Text(text = usuario.name, fontWeight = FontWeight.Bold)
                            Text(text = usuario.email, color = Color.Gray)
                        }
                    }
                }
            }
        }
    }
}
```

### **collectAsState() = a ponte com a UI**

`collectAsState()` observa o `StateFlow` do `ViewModel` e recompõe a tela automaticamente quando o valor muda — exatamente como o React re-renderiza ao mudar o `useState`. O `when (estado)` substitui o `if (loading)` / `if (error)` / `return` do React, com a vantagem de o compilador Kotlin garantir que todos os casos da sealed class sejam tratados.

## 4. Permissões em Tempo de Execução

Assim como no Expo, todo acesso a recursos sensíveis do dispositivo exige permissão explícita do usuário. No Android nativo, esse fluxo é gerenciado com `ActivityResultContracts`.

### 4.1 Declarando no Manifest

Toda permissão usada deve ser declarada em `AndroidManifest.xml` — equivalente às configurações do `app.json` no Expo:

```
<!-- AndroidManifest.xml -->
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

### 4.2 Solicitando Permissão em Compose

```
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.core.content.ContextCompat
import android.content.pm.PackageManager
import android.Manifest

@Composable
fun TelaComPermissao() {
    val context = LocalContext.current
    var permissaoConcedida by remember {
        mutableStateOf(
            ContextCompat.checkSelfPermission(
                context, Manifest.permission.CAMERA
            ) == PackageManager.PERMISSION_GRANTED
        )
    }

    val launcher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.RequestPermission()
    ) { concedida -> permissaoConcedida = concedida }

    Button(onClick = {
        if (!permissaoConcedida) {
            launcher.launch(Manifest.permission.CAMERA)
        }
    }) {
        Text(if (permissaoConcedida) "Permissão concedida" else "Solicitar câmera")
    }
}
```

#### Comparação com o Expo

`rememberLauncherForActivityResult` → equivalente a `requestPermissionsAsync()` do expo-\*  
`PackageManager.PERMISSION_GRANTED` → equivalente à comparação com `status === 'granted'`  
A diferença estrutural: no Android nativo, o launcher precisa ser registrado ANTES de ser chamado (boa prática de ciclo de vida); no Expo isso é abstraído pela própria função `async`.

## 5. Câmera e Galeria

---

### 5.1 Abrindo a Galeria de Fotos

```
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import android.net.Uri
import coil.compose.AsyncImage

@Composable
fun SeletorImagem() {
    var imagemUri by remember { mutableStateOf<Uri?>(null) }

    val launcher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.PickVisualMedia()
    ) { uri -> imagemUri = uri }

    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Box(
            modifier = Modifier
                .size(120.dp)
                .clip(CircleShape)
                .background(Color.LightGray)
                .clickable {
                    launcher.launch(
                        PickVisualMediaRequest(ActivityResultContracts.PickVisualMedia.ImageOnly)
                    )
                },
            contentAlignment = Alignment.Center
        ) {
            if (imagemUri != null) {
                AsyncImage(
                    model = imagemUri,
                    contentDescription = "Foto selecionada",
                    modifier = Modifier.fillMaxSize().clip(CircleShape)
                )
            } else {
                Text("Selecionar foto")
            }
        }
    }
}
```

#### Dependência necessária

implementation("io.coil-kt:coil-compose:2.6.0") // exibição de imagens (equivalente ao Image do RN)

### 5.2 Abrindo a Câmera

```
import androidx.activity.result.contract.ActivityResultContracts

val launcherCamera = rememberLauncherForActivityResult(
    contract = ActivityResultContracts.TakePicturePreview()
) { bitmap ->
```

```

        bitmap?.let {
            // bitmap é a foto capturada (android.graphics.Bitmap)
            imagemBitmap = it
        }
    }

    Button(onClick = { launcherCamera.launch() }) {
        Text("Tirar foto")
    }
}

```

| Expo / React Native                                | Android Nativo (Compose)                     |
|----------------------------------------------------|----------------------------------------------|
| <b>expo-image-picker (launchImageLibraryAsync)</b> | ActivityResultContracts.PickVisualMedia()    |
| <b>expo-image-picker (launchCameraAsync)</b>       | ActivityResultContracts.TakePicturePreview() |
| <b>resultado.assets[0].uri</b>                     | uri (Galeria) ou bitmap (Câmera)             |
| <b>Image source={{ uri }}</b>                      | AsyncImage(model = uri) — biblioteca Coil    |

## 6. Localização (GPS)

### 6.1 Dependência

```
implementation("com.google.android.gms:play-services-location:21.3.0")
```

### 6.2 Obtendo a Posição Atual

```

import com.google.android.gms.location.LocationServices
import com.google.android.gms.location.Priority

@Composable
fun TelaLocalizacao() {
    val context = LocalContext.current
    var localizacao by remember { mutableStateOf<Pair<Double, Double>?>(null) }

    val fusedLocationClient = remember {
        LocationServices.getFusedLocationProviderClient(context)
    }

    val launcherPermissao = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.RequestPermission()
    ) { concedida ->
        if (concedida) {
            fusedLocationClient.getCurrentLocation(
                Priority.PRIORITY_HIGH_ACCURACY, null
            ).addOnSuccessListener { location ->
                location?.let {
                    localizacao = Pair(it.latitude, it.longitude)
                }
            }
        }
    }
}

```

```

    }

    Column(modifier = Modifier.padding(16.dp)) {
        Button(onClick = {
            launcherPermissao.launch(Manifest.permission.ACCESS_FINE_LOCATION)
        }) {
            Text("Obter localização")
        }

        localizacao?.let { (lat, lon) ->
            Text("Latitude: $lat")
            Text("Longitude: $lon")
        }
    }
}

```

| Expo / React Native                                 | Android Nativo (Compose)                                        |
|-----------------------------------------------------|-----------------------------------------------------------------|
| <b>Location.requestForegroundPermissionsAsync()</b> | ActivityResultContracts.RequestPermission()                     |
| <b>Location.getCurrentPositionAsync()</b>           | fusedLocationClient.getCurrentLocation()                        |
| <b>Location.watchPositionAsync()</b>                | fusedLocationClient.requestLocationUpdates()                    |
| <b>subscription.remove() (cleanup)</b>              | fusedLocationClient.removeLocationUpdates() no DisposableEffect |

### Cleanup com DisposableEffect

Assim como no `useEffect` com `cleanup`, o rastreamento contínuo de localização deve ser interrompido ao saída da tela. Em Compose, isso é feito com `DisposableEffect`, que possui um bloco `onDispose { }` equivalente ao `return () => { ... }` do React.

## 7. Armazenamento Local com DataStore

O `DataStore` é a evolução do antigo `SharedPreferences` e é o equivalente direto do `AsyncStorage` do React Native.

### 7.1 Dependência

```
implementation("androidx.datastore:datastore-preferences:1.1.1")
```

### 7.2 Configurando e Usando

```

// data/PreferenciasDataStore.kt
import androidx.datastore.preferences.core.*
import androidx.datastore.preferences.preferencesDataStore
import kotlinx.coroutines.flow.map

val Context.dataStore by preferencesDataStore(name = "configuracoes")

```



```

object PreferenciasKeys {
    val TEMA = stringPreferencesKey("tema")
    val TOKEN = stringPreferencesKey("token")
}

class PreferenciasRepository(private val context: Context) {

    val tema: Flow<String> = context.dataStore.data
        .map { preferencias -> preferencias[PreferenciasKeys.TEMA] ?: "light" }

    suspend fun salvarTema(tema: String) {
        context.dataStore.edit { preferencias ->
            preferencias[PreferenciasKeys.TEMA] = tema
        }
    }
}

```

| Expo / React Native                       | Android Nativo (Compose)                        |
|-------------------------------------------|-------------------------------------------------|
| <b>AsyncStorage.setItem(chave, valor)</b> | dataStore.edit { it[chave] = valor }            |
| <b>AsyncStorage.getItem(chave)</b>        | dataStore.data.map { it[chave] } (retorna Flow) |
| <b>expo-secure-store</b>                  | EncryptedSharedPreferences ou Jetpack Security  |
| <b>useStorage (hook customizado)</b>      | Repository pattern + collectAsState()           |

## 8. Notificações Locais

### 8.1 Canal de Notificação (obrigatório desde Android 8)

```

import android.app.NotificationChannel
import android.app.NotificationManager
import android.os.Build

fun criarCanalNotificacao(context: Context) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val canal = NotificationChannel(
            "canal_principal",
            "Notificações do App",
            NotificationManager.IMPORTANCE_DEFAULT
        )
        val manager = context.getSystemService(NotificationManager::class.java)
        manager.createNotificationChannel(canal)
    }
}

```

### 8.2 Disparando uma Notificação

```

import androidx.core.app.NotificationCompat
import androidx.core.app.NotificationManagerCompat

```

```

fun enviarNotificacao(context: Context, titulo: String, mensagem: String) {
    val notificacao = NotificationCompat.Builder(context, "canal_principal")
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle(titulo)
        .setContentText(mensagem)
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .build()

    NotificationManagerCompat.from(context).notify(1, notificacao)
}

// Uso em um Composable, após uma ação do usuário:
Button(onClick = {
    enviarNotificacao(context, "Pedido confirmado!", "Seu pedido #1234 foi recebido.")
}) {
    Text("Confirmar pedido")
}

```

### Permissão obrigatória a partir do Android 13

POST\_NOTIFICATIONS deve ser solicitada em tempo de execução, da mesma forma que Notifications.requestPermissionsAsync() é exigido no Expo.

## 9. Comparativo Geral — Bibliotecas e Conceitos

| Expo / React Native        | Android Nativo                              | Finalidade                                         |
|----------------------------|---------------------------------------------|----------------------------------------------------|
| fetch / hooks customizados | Retrofit + Coroutines + ViewModel           | Chamadas HTTP e gerenciamento de estado assíncrono |
| useState / useEffect       | mutableStateOf / LaunchedEffect / StateFlow | Estado reativo da UI                               |
| Context API                | ViewModel + Hilt (injeção de dependência)   | Compartilhamento de estado entre telas             |
| expo-image-picker          | ActivityResultContracts (Pick/TakePicture)  | Acesso a câmera e galeria                          |
| expo-location              | FusedLocationProviderClient                 | Geolocalização                                     |
| AsyncStorage               | DataStore Preferences                       | Armazenamento local não sensível                   |
| expo-secure-store          | EncryptedSharedPreferences                  | Armazenamento local sensível (tokens)              |
| expo-notifications         | NotificationManager + NotificationCompat    | Notificações locais                                |
| Image (componente)         | AsyncImage (biblioteca Coil)                | Carregamento e exibição de imagens remotas         |